kaggle
Search
Create
Home
Competitions
Datasets
Models
Code
Discussions
Learn
More
Your Work
RECENTLY VIEWED
Text Classification wit...
Simple Tweets classifi...
BERT Text Classificatio...
Natural Language Proc...
Simple Transfer Learni...
RECENTLY EDITED
VGG16/19, ResNet50
TK HCD VGG16 v01
[TF Demo] Image capti...
Image Captioning by E...
BOOKMARKS
Histopathologic Cance...

Text Classification with GloVe, LSTM/GRU(99% acc)

Tol Notebook Input Output Logs Comments (3)
3
Copy & Edit 56

Table of Contents
A Step by Step Guide to Text...
Solution Framework
Model Overview
Loading data and visualizing
Combine fake and true dataframes
Text Cleaning
Prepare train and test datasets
Word embedding using pre-...
Modeling

In most of the NLP tasks, we need to represent each word in the text with an integer value (index) before feeding it to any model. In this way, the text will be converted to a sequence of integer values. One of the ways of doing that is with Keras. Keras provides an API for tokenizing the text. Tokenizer in Keras finds the frequency of each unique word and sort them based on their frequency. It then assigns an integer value starting from 1 to each word from the top. You can see the index mapping dictionary by reading tokenizer.word_index.

There are two distinct steps in tokenizing in this way:

1. fit_on_texts: We'll fit the tokenizer on our training data to create the word indices
2. texts_to_sequences: using the word index dictionary from step above, we take this step to transform both train and test data.

Here, we set num_words to a limit such as 10000 words. num_words is a parameter that defines the maximum number of words to keep, based on the word frequency. Keras actually keeps (num_words-1) words. We can leave the num_words to 'None' and tokenizer will pick all the words in the vacabulary.

Padding and truncating the input training sequences

All your input sequences to the model need to have the same length. In order to achieve that, we can use a function that pads the short sequences with zeros (options are 'pre' or 'post' which pads either before or after each sequence). It also truncates any sequence that is longer than a predefined parameter "maxlen". Truncating also has the option of 'pre' or 'post' which either truncates at the beginning or at the end of the sequences.

In [9]:
```
max_words = 10000
tokenizer = text.Tokenizer(num_words = max_words)
# create the vocabulary by fitting on x_train text
tokenizer.fit_on_texts(xtrain)
# generate the sequence of tokens
xtrain_seq = tokenizer.texts_to_sequences(xtrain)
xtest_seq = tokenizer.texts_to_sequences(xtest)

# pad the sequences
xtrain_pad = sequence.pad_sequences(xtrain_seq, maxlen=max_len)
xtest_pad = sequence.pad_sequences(xtest_seq, maxlen=max_len)
word_index = tokenizer.word_index

print('text example:', xtrain[0])
print('sequence of indices(before padding):', xtrain_seq[0])
print('sequence of indices(after padding):', xtrain_pad[0])
```

```
text example: As US budget fight looms Republicans flip fiscal script
sequence of indices(before padding): [47, 2455, 615, 996, 3704, 118, 2692, 3534]
sequence of indices(after padding): [   0    0    0    0    0    0    0    0    0    0    0    0
0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0    0    0    0    0    0
    0    0    0    0    0    0    0    0    0   47 2455  615  996 3704
  118 2692 3534]
```

Word embedding using pre-trained GloVe vectors

Now that we have tokenized the text, we use GloVe pretrained vectors. Word embeddings is a way to represent words with similar meaning to have a similar representation. Using word embedding through GloVe, we can have a decent performance with models with even relatively small label training sets.

You can download GloVe pre-trained word vectors from the link below. There are different sizes of vocabulary and embedding dimension available.

https://nlp.stanford.edu/projects/glove/ (https://nlp.stanford.edu/projects/glove/)

Load the GloVe vectors

In [10]:
```
%%time
embedding_vectors = {}
# with open('/kaggle/input/glove6b100d/glove.6B.100d.txt','r',encoding='utf-8') as file:
with open('/kaggle/input/glove42b300dtxt/glove.42B.300d.txt','r',encoding='utf-8') as file:
    for row in file:
        values = row.split(' ')
        word = values[0]
        weights = np.asarray([float(val) for val in values[1:]])
        embedding_vectors[word] = weights
print(f"Size of vocabulary in GloVe: {len(embedding_vectors)}")
```

```
Size of vocabulary in GloVe: 1917494
CPU times: user 4min 21s, sys: 6.44 s, total: 4min 27s
Wall time: 4min 27s
```

Create an embedding matrix with the GloVe vectors

The embedding matrix has a shape of (vocabulary length, embedding dimension).

Note tht vacab_len is equal to max_words if we set that limit when tokenizing. Otherwise, vocab_len is equal to lenght of all words in word_index+1.

Each row of the embedding matrix belongs to one word in the vocabulary (derived from xtrain) and it contains the weights of embedding vector of that word.

In the code below, we initialze the embedding matrix with zeros. If a word in our word_index is not found in the embedding vectors from GloVe.

The weight of that word remains as zero. Below, you can see print out of the some example words that are out of vacabulary (OOV).

In [11]:
```
#initialize the embedding_matrix with zeros
emb_dim = 300
if max_words is not None:
    vocab_len = max_words
else:
```

View Active Events